

Deploy Guide

Rama LiveKit Service — UAT Deployment Guide

คู่มือติดตั้งและรัน `docker-compose.yaml`

Stack: Redis 7 + LiveKit Server v1.13 + rama-livekit-service (Bun)

1. สิ่งที่ต้องเตรียมบนเครื่อง Prod

รายการ	รายละเอียด
Docker Engine	≥ 24.0 พร้อม <code>docker compose</code> plugin
OS	Linux (แนะนำ Ubuntu 22.04 หรือ 24.04)
Ports ที่ต้องเปิด	<code>7880</code> (LiveKit signaling), <code>8080</code> (API), <code>50000-60000/udp</code> (WebRTC ICE)
Firewall	ปิดการเข้าถึง <code>7880</code> และ <code>8080</code> จากภายนอกถ้ามี reverse proxy
Docker image	<code>harbor.rama.mahidol.ac.th/livekit/rama-livekit-service:<tag></code>

2. ไฟล์ที่ต้องมีใน working directory

```
rama-livekit/      # (หรือ path ที่ต้องการ)
├── .env            # สร้างใหม่ (ดูหัวข้อ 3)
├── redis.conf      # สร้างใหม่ (ดูหัวข้อ 4)
├── livekit.yaml    # สร้างใหม่ (ดูหัวข้อ 5)
├── docker-compose.yaml # สร้างใหม่ (ดูหัวข้อ 6)
└── redis_data/    # auto-create ตอนรันครั้งแรก
```

3. ไฟล์ `.env`

```
NODE_ENV=uat
PORT=8080
# Docker image name for docker-compose.yaml (e.g. hosaprama/rama-livekit-service)
HOSAPP_IMAGE=harbor.rama.mahidol.ac.th/livekit/rama-livekit-service
IMAGE_TAG=v1.0.0-uat
# Timezone (default UTC)
TZ=Asia/Bangkok
# JWT secrets (required in production)
JWT_ACCESS_SECRET=<your_jwt_access_secret_here>
JWT_REFRESH_SECRET=<your_jwt_refresh_secret_here>
# JWT expiry (optional overrides)
JWT_ACCESS_EXP=15m
JWT_REFRESH_EXP=7d
# LiveKit
LIVEKIT_API_KEY=<your_livekit_api_key_here>
LIVEKIT_API_SECRET=<your_livekit_api_secret_here>
MAX_PARTICIPANTS=10
```

```

EMPTY_TIMEOUT=600
ACCESS_TOKEN_TTL=3h
# When LiveKit runs alongside this service (docker-compose.yaml, network_mode: host):
LIVEKIT_SERVER_URL=wss://127.0.0.1:7880
LIVEKIT_HOST_URL=http://127.0.0.1:7880
# Redis
REDIS_URL=redis://127.0.0.1:6379
# Redis password used by docker-compose.yaml's redis service (see redis.conf)
REDIS_PASSWORD=<your_redis_password_here>
# S3 / MinIO
S3_ACCESS_KEY=<your_s3_access_key_here>
S3_SECRET_KEY=<your_s3_secret_key_here>
S3_REGION=<your_s3_region>
S3_BUCKET=<your_s3_bucket_name>
S3_ENDPOINT=<https://your_s3_endpoint_url>
S3_FORCE_PATH_STYLE=true
# S3 presigned URL expiration in hours (default 1)
S3_URL_EXPIRATION_HOURS=1
# Default upload quality (optional)
DEFAULT_QUALITY=high
# Recording layout: grid | speaker | single-speaker (default grid)
EGRESS_LAYOUT=grid
# Optional custom egress template base URL (object-fit: contain, show camera-off).
# Leave empty to use LiveKit's built-in template.
EGRESS_TEMPLATE_URL=
# Max concurrent egress recordings (default 1)
EGRESS_CAPACITY_TOTAL=1

```

ค่าที่ต้องแก้ไขเป็นค่าจริง:

Variable	ตัวอย่าง	หมายเหตุ
<code>RAMA_LIVEKIT_SERVICE_IMAGE</code>	<code>hosapprama/rama-livekit-service</code>	ชื่อ image บนrama registry <code>harbor.rama.mahidol.ac.th/livekit/rama-livekit-service</code>
<code>IMAGE_TAG</code>	<code>v1.0.0-uat</code>	tag ที่ build จาก Jenkins
<code>JWT_ACCESS_SECRET</code>	Base64url secret 32-byte	จำเป็น — ใช้ <code>openssl rand -base64 32 tr '+/' '-_' tr -d '='</code>
<code>JWT_REFRESH_SECRET</code>	Base64url secret 32-byte	จำเป็น — ใช้ <code>openssl rand -base64 32 tr '+/' '-_' tr -d '='</code>
<code>LIVEKIT_API_KEY</code>	<code>APIxxxxxxxxxxx</code>	ต้องตรงกับ <code>livekit.yaml</code> ใช้ <code>echo "API\$(openssl rand -hex 6)"</code>
<code>LIVEKIT_API_SECRET</code>	(random 32+ chars)	ต้องตรงกับ <code>livekit.yaml</code> ใช้ <code>openssl rand -base64 32 tr '+/' '-_' tr -d '='</code>
<code>REDIS_PASSWORD</code>	(random)	ต้องตรงกับ <code>redis.conf</code> ใช้ <code>LC_ALL=C tr -dc 'A-Za-z0-9' </dev/urandom head -c 32; echo</code>

Variable	ตัวอย่าง	หมายเหตุ
REDIS_URL	redis://:password@127.0.0.1:6379	ใส่ password ด้วย
S3_ACCESS_KEY S3_SECRET_KEY S3_REGION S3_BUCKET S3_ENDPOINT S3_FORCE_PATH_STYLE S3_URL_EXPIRATION_HOURS	S3_ACCESS_KEY=<your_s3_access_key_here> S3_SECRET_KEY=<your_s3_secret_key_here> S3_REGION=<your_s3_region> S3_BUCKET=<your_s3_bucket_name> S3_ENDPOINT=<https://your_s3_endpoint_url> S3_FORCE_PATH_STYLE=true S3_URL_EXPIRATION_HOURS=1	MinIO, SeaweedFS หรือ S3 จริง

4. ไฟล์ `redis.conf`

```
# Bind ที่ loopback (LiveKit รันบน host เดียวกัน)
bind 127.0.0.1

# Persistence
dir /data
save 900 1
save 300 10
save 60 10000

# Memory policy - evict least recently used keys
maxmemory 1gb
maxmemory-policy noeviction

# Logging
loglevel notice
```

5. ไฟล์ `livekit.yaml`

API key/secret และ Redis password ดึงจาก `.env` ผ่าน env var แล้ว

(`LIVEKIT_KEYS`, `REDIS_PASSWORD` ใน docker-compose)

****ไม่ต้องใส่ `api_key`, `api_secret`, `redis.password` ในไฟล์นี้****

```
# LiveKit Server config
# Docs: https://docs.livekit.io/server/config/

port: 7880
bind_addresses:
  - "127.0.0.1"

# Redis address (password ส่งผ่าน env var REDIS_PASSWORD)
redis:
  address: "127.0.0.1:6379"

# Room defaults
room:
  auto_create: true
  empty_timeout: 600
  max_participants: 10
```

```
# WebRTC / RTC
rtc:
  tcp_port:7881
  port_range_start:50000
  port_range_end:60000
  use_external_ip:true
  enable_loopback_candidate:false

# Logging
log_level: info

# TURN server (optional – แนะนำถ้ามี client หลัง NAT strict)
# turn:
#   enabled: true
#   domain: turn.yourdomain.com
#   tls_port: 5349
#   secret: "YOUR_TURN_SECRET"
```

ถ้ารันบน cloud VM เปิด UDP 50000-60000 ใน security group และตั้ง `rtc.use_external_ip: true`

6. ไฟล์ `docker-compose.yaml`

```
name: livekit-core

services:
  redis:
    image: redis:7-alpine
    container_name: livekit-redis
    # Password ส่งผ่าน --requirepass (ดึงจาก .env) ไม่ต้องใส่ใน redis.conf
    command: sh -c 'redis-server /etc/redis.conf --requirepass "$$REDIS_PASSWORD"'
    restart: unless-stopped
    network_mode: host # binds 127.0.0.1 + the private IP (redis.conf)
    environment:
      - REDIS_PASSWORD=${REDIS_PASSWORD}
    volumes:
      - ./redis.conf:/etc/redis.conf:ro
      - ./redis_data:/data
    healthcheck:
      test: ["CMD-SHELL", "redis-cli -a \"$$REDIS_PASSWORD\" --no-auth-warning ping | grep -q PONG"]
      interval: 10s
      timeout: 5s
      retries: 5

  livekit:
```

```

image: livekit/livekit-server:v1.13
container_name: livekit-server
command: --config /etc/livekit.yaml
restart: unless-stopped
network_mode: host
# LiveKit รับ API key/secret และ Redis password ผ่าน env var โดยตรง
# ไม่ต้องใส่ค่าเหล่านี้ใน livekit.yaml
environment:
  - LIVEKIT_KEYS=${LIVEKIT_API_KEY}:${LIVEKIT_API_SECRET}
  - REDIS_PASSWORD=${REDIS_PASSWORD}
volumes:
  - ./livekit.yaml:/etc/livekit.yaml:ro
depends_on:
  redis:
    condition: service_healthy

livekit-service:
image: ${HOSAPP_IMAGE:?set HOSAPP_IMAGE in .env}:${IMAGE_TAG:-latest}
container_name: rama-livekit-service
restart: unless-stopped
pull_policy: always
network_mode: host
env_file:
  - .env
environment:
  NODE_ENV: production
  PORT: 8080
depends_on:
  redis:
    condition: service_healthy
  livekit:
    condition: service_started
healthcheck:
  test: ["CMD-SHELL", "bun -e 'fetch(\"http://localhost:8080/health-check\").then(r=>process.exit(r.ok?0:1)).catch(()=>process.exit(1))'"]
  interval: 15s
  timeout: 5s
  retries: 5
  start_period: 20s

```

7. ขั้นตอน Deploy

```

# 1. เข้า working directory ตาม path ที่ตั้งไว้ เช่น
cd rama-livekit

# 2. สร้างไฟล์ config ทั้งหมด (ตามหัวข้อ 3-6)
#   .env, redis.conf, livekit.yaml, docker-compose.yaml

```

```
# 3. Pull image ล่าสุด
docker compose -f docker-compose.yaml pull

# 4. รัน (detached)
docker compose -f docker-compose.yaml up -d

# 5. ตรวจสอบสถานะ
docker compose -f docker-compose.yaml ps

# 6. ตรวจสอบ logs (รอสัก 20 วินาทีให้ healthcheck ผ่าน)
docker compose -f docker-compose.yaml logs -f rama-livekit-service
```

8. ตรวจสอบว่ารันสำเร็จ

```
# 3 containers ต้อง status "Up" และ "healthy"
docker compose -f docker-compose.yaml ps

# API health check
curl http://localhost:8080/health-check
# ควรได้: {"code":200,"success":true,...}

# Redis ตอบ
docker exec livekit-redis redis-cli -a "$REDIS_PASSWORD" --no-auth-warning ping
# ควรได้: PONG

# LiveKit ตอบ
curl http://localhost:7880/
# ควรได้: 404 หรือ response จาก LiveKit (ไม่ใช่ connection refused)
```

9. อัปเดตเวอร์ชันใหม่

```
# เมื่อ build image tag ใหม่จาก Jenkins แล้ว
# 1. แก้ IMAGE_TAG ใน .env
# IMAGE_TAG=v1.1.0-uat

# 2. Pull + restart เฉพาะ service
docker compose -f docker-compose.yaml pull rama-livekit-service
docker compose -f docker-compose.yaml up -d rama-livekit-service

# 3. ตรวจสอบ
docker compose -f docker-compose.yaml logs -f rama-livekit-service
```

10. การย้อนกลับ (Rollback)

```
# 1. แก้ IMAGE_TAG กลับเป็นเวอร์ชันเดิม
# IMAGE_TAG=v1.0.0-uat

# 2. Pull + restart
docker compose -f docker-compose.yaml pull rama-livekit-service
docker compose -f docker-compose.yaml up -d rama-livekit-service
```

11. ปัญหาที่พบบ่อย

อาการ	สาเหตุ	วิธีแก้
redis container unhealthy	password ใน <code>.env</code> ไม่ตรงกับ <code>redis.conf</code>	ตรวจสอบ <code>REDIS_PASSWORD</code> ทั้ง 2 ไฟล์
rama-livekit-service ติด <code>LIVEKIT_API_KEY</code>	key/secret ไม่ตรงกับ <code>livekit.yaml</code>	ตรวจสอบให้ตรงกันทั้ง 3 ที่
ไม่สามารถเชื่อมต่อวิดีโอได้	UDP <code>50000-60000</code> ถูก block	เปิดใน firewall / security group
<code>EADDRINUSE :8080</code>	port ชนกับ service อื่น	เปลี่ยน <code>PORT</code> ใน <code>.env</code>
container รันแป๊บแล้ว crash	ขาด env var ที่จำเป็น	ดู <code>docker logs rama-livekit-service</code>

12. โครงสร้าง Network

